

BRK (00)		Cycles: 7	Size: 2 *
Implied			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	** Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand.	
2.1		Inc. PC .	
2.2	*** Write to u8(S.L +0) \$0100	Push PC.H onto stack.	
3.1			
3.2	*** Write to u8(S.L -1) \$0100	Push PC.L onto stack.	
4.1		Use Interrupt flags to decide which vector to use. RESET = \$FFFC. NMI = \$FFFA. IRQ/BRK = \$FFFE.	
4.2	*** Write to u8(S.L -2) \$0100	Push P onto stack with B flag if software BRK.	
5.1		Dec. S.L by 3 and set S.H to 1.	
5.2	Read Vector	Store to Address.L.	
6.1		Set I , unset D .	
6.2	Read Vector + 1	Store to Address.H.	
7.1		Enable writes to PC (disabled by NMI/IRQ). Copy Address to PC. Set PB to 0.	
7.2	Read PC:PB	Store as OpCode.	
+X.1			

* Concerning the BRK instruction, you should also note that although its second byte is basically a “don’t care” byte – that is, it can have any value - the BRK (and COP instruction as well) is a two-byte instruction, the second byte sometimes is used as a signature byte to determine the nature of the BRK being executed. When an RTI instruction is executed, control always returns to the second byte past the BRK opcode. — assembly-programming-manual-for-w65c816.pdf

** If NMI, IRQ, or RESET, OpCode is forced to 0 and writes to PC are disabled.

*** If handling a RESET, the writes turn into reads. The databus is populated but ignored.

ORA (01) Cycles: 6		Size: 2
Direct Indexed Indirect (d,x) (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address.L = Pointer.L. Address.H = D .H.
2.2		If D .L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:DB	Store as Operand.
7.1		Perform A = Operand. Set N based off (A .L & \$80) and Z based off A .L.
7.2	Read PC:PB	Store as OpCode.
+X.1		

COP (02)	Cycles: 7	Size: 2
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Write to u8(S .L+0) \$0100	Push PC .H onto stack.
3.1		
3.2	Write to u8(S .L-1) \$0100	Push PC .L onto stack.
4.1		Set Vector to \$FFF4.
4.2	Write to u8(S .L-2) \$0100	Push P onto stack.
5.1		Dec. S .L by 3 and set S .H to 1.
5.2	Read Vector	Store to Address.L.
6.1		Set I , unset D .
6.2	Read Vector + 1	Store to Address.H.
7.1		Copy Address to PC . Set PB to 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (03)	Cycles: 4	Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Perform A = Operand. Set N based off (A .L & \$80) and Z based off A .L.
4.2	Read PC:PB	Store as OpCode.
+X.1		

TSB (04)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set Z based off (A.L & Operand.L). Perform Operand = A .
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (05)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

ASL (06)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set C based off (Operand.L & \$80). Perform Operand.L <=< 1. Set N based off (Operand.L & \$80) and Z based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (07)	Cycles: 6	Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

PHP (08)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Write to ((S .L \$0100)+0)	Inc. PC .
1.2		
2.1		Set Operand.L to P with X and M flags set.
2.2		Push Operand.L onto stack.
3.1		Dec. S .L by 1 and set S .H to 1.
3.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (09)		Cycles: 2	Size: 2
Immediate			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Operand.	
2.1		Inc. PC . Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .	
2.2		Store as OpCode.	
+X.1			

ASL (0A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set C based off (A.L & \$80). Perform A.L <= 1. Set N based off (A.L & \$80) and Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

PHD (0B)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Write to u8(S.L +0) \$0100 Write to ((S.L \$0100)-1)	Inc. PC .
1.2		
2.1		
2.2		Push D.H onto stack.
3.1		
3.2		Push D.L onto stack.
4.1		
4.2		Store as OpCode.
+X.1		

TSB (0C)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		
4.2	Write to Address: DB	Write Operand.L.
5.1		Set Z based off (A .L & Operand.L). Perform Operand = A .
5.2	Write to Address: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (0D)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Perform A = Operand. Set N based off (A .L & \$80) and Z based off A .L.
4.2	Read PC:PB	Store as OpCode.
+X.1		

ASL (0E)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		
4.2	Write to Address: DB	Write Operand.L.
5.1		Set C based off (Operand.L & \$80). Perform Operand.L <=< 1. Set N based off (Operand.L & \$80) and Z based off Operand.L.
5.2	Write to Address: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (0F)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand.
5.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

BPL (10)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC . Decide to branch if N is 0.
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set PC .H to Address.H
4.2		Store as OpCode.
+X.1		

ORA (11)		Cycles: 5	Size: 2
Direct Indirect Indexed (d),y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).	
5.2			
6.1			
6.2	Read Pointer:Bank	Store as Operand.	
7.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .	
7.2	Read PC:PB	Store as OpCode.	
+X.1			

ORA (12)		Cycles: 5	Size: 2
Direct Indirect (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Pointer: DB	Store as Operand.	
6.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ORA (13)		Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB .	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + (S .L \$100.)	
2.2			
3.1			
3.2		Store as Pointer.	
4.1			
4.2		Store as Pointer.H.	
5.1		Perform Tmp = ui32(Pointer + Y).	
		Pointer = ui16(Tmp).	
5.2	Read Address	Bank = DB + (Tmp >> 16).	
6.1			
6.2		Store as Operand.	
7.1		Perform A = Operand. Set N based off (A .L & \$80) and Z based off A .L.	
7.2	Read PC:PB .	Store as OpCode.	
+X.1			

TRB (14)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set Z based off (A.L & Operand.L). Perform Operand &= ~ A .
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (15)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Read PC:PB	Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2		Store as OpCode.
+X.1		

ASL (16)	Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Write to Address	Write Operand.L.
5.2		Set C based off (Operand.L & \$80). Perform Operand.L <=< 1. Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.1		
6.2	Write to Address	Write Operand.L.
7.1	Read PC:PB	
7.2		Store as OpCode.
+X.1		

ORA (17) Cycles: 6		Size: 2
Direct Indirect Indexed Long [d],y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

CLC (18)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set the C flag to 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (19) Cycles: 4		Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB .	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2	Read PC:PB .	Store as OpCode.
+X.1		

INC (1A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform A.L += 1, set N based off (A.L & \$80), set Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

TCS (1B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set S.L to A.L and S.H to 1.
2.2	Read PC:PB	Store as OpCode.
+X.1		

TRB (1C)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		
4.2	Write to Address: DB	Write Operand.L.
5.1		Set Z based off (A .L & Operand.L). Perform Operand &= ~ A .
5.2	Write to Address: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

ORA (1D) Cycles: 4		Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB .	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform A = Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2	Read PC:PB .	Store as OpCode.
+X.1		

ASL (1E)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB .	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set C based off (Operand.L & \$80). Perform $\text{Operand.L} \ll= 1$. Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB .	Store as OpCode.
+X.1		

ORA (1F) Cycles: 5		Size: 4
Absolute Long, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB .	Store as Pointer.H.
3.1		Inc. PC .
3.2	Read PC:PB .	Stores as Bank.
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank += ($\text{Tmp} \gg 16$).
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform $\text{A} \mid = \text{Operand}$. Set N based off (A.L & \$80) and Z based off A.L .
5.2	Read PC:PB .	Store as OpCode.
+X.1		

JSR (20)	Cycles: 6	Size: 3
Absolute		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read PC:PB	Discard.
4.1		Dec. PC .
4.2	Write to u8(S .L+0) \$0100	Push PC .H onto stack.
5.1		
5.2	Write to u8(S .L-1) \$0100	Push PC .L onto stack.
6.1		Dec. S .L by 2 and set S .H to 1. Perform PC = Address.
6.2	Read PC:PB	Store as OpCode.
+X.1		

AND (21)		Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB .	Store as Pointer.	
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address.L = Pointer.L. Address.H = D .H.	
2.2		If D .L is 0, skip the next cycle.	
3.1		Address.H = Pointer.H (fixes high byte of address).	
3.2			
4.1			
4.2	Read Address	Store as Pointer.	
5.1			
5.2	Read Address + 1	Store as Pointer.H.	
6.1			
6.2	Read Pointer: DB	Store as Operand.	
7.1		Perform A &= Operand. Set N based off (A .L & \$80) and Z based off A .L.	
7.2	Read PC:PB .	Store as OpCode.	
+X.1			

JSL (22)	Cycles: 8	Size: 4
Absolute Long		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Write to u8(S .L+0) \$0100	Push PB onto stack.
4.1		
4.2	Read u8(S .L+0) \$0100	Discard.
5.1		
5.2	Read PC:PB	Stores as Bank.
6.1		
6.2	Write to ((S .L \$0100)-1)	Push PC .H onto stack.
7.1		
7.2	Write to ((S .L \$0100)-2)	Push PC .L onto stack.
8.1		Dec. S .L by 3 and set S .H to 1. Copy Address to PC . Copy Bank to PB .
8.2	Read PC:PB	Store as OpCode.
+X.1		

AND (23)	Cycles: 4	Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Perform A &= Operand. Set N based off (A .L & \$80) and Z based off A .L.
4.2	Read PC:PB .	Store as OpCode.
+X.1		

BIT (24)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Set Z based off (A.L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40).
4.2	Read PC:PB	Store as OpCode.
+X.1		

AND (25)		Cycles: 3	Size: 2
Direct (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand.	
4.1		Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .	
4.2	Read PC:PB	Store as OpCode.	
+X.1			

ROL (26)		Cycles: 5	Size: 2
Direct (Read/Modify/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand.	
4.1			
4.2	Write to Address	Write Operand.L.	
5.1		Set C based off (Operand.L & \$80). Perform Operand.L = (Operand.L << 1) C . Set N based off (Operand.L & \$80) and Z based off Operand.L.	
5.2	Write to Address	Write Operand.L.	
6.1			
6.2	Read PC:PB	Store as OpCode.	
+X.1			

AND (27)	Cycles: 6	Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

PLP (28)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S.L by 1 and set S.H to 1.
3.2		Store as Operand.L.
4.1	Read S.L \$0100	P = Operand.L, set X and M flags, set X.H and Y.H to 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

AND (29)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

ROL (2A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set C based off (A.L & \$80). Perform A.L = (A.L << 1) C . Set N based off (A.L & \$80) and Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

PLD (2B)	Cycles: 5	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read ((S.L \$0100)+1)	Inc. PC .
1.2		
2.1		
2.2		
3.1		
3.2		Store as Operand.L.
4.1		
4.2		Store as Operand.H.
5.1		Inc. S.L by 2 and set S.H to 1. Set D to Operand. Set N based off (D.H & \$80). Set Z based off D .
5.2		Store as OpCode.
+X.1		

BIT (2C)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Set Z based off (A .L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40).
4.2	Read PC:PB	Store as OpCode.
+X.1		

AND (2D)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Perform A &= Operand. Set N based off (A .L & \$80) and Z based off A .L.
4.2	Read PC:PB	Store as OpCode.
+X.1		

ROL (2E)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		
4.2	Write to Address: DB	Write Operand.L.
5.1		Set C based off (Operand.L & \$80). Perform Operand.L = (Operand.L << 1) C . Set N based off (Operand.L & \$80) and Z based off Operand.L.
5.2	Write to Address: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

AND (2F)		Cycles: 5	Size: 4
Absolute Long (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Address.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Address.L.	
3.1		Inc. PC .	
3.2	Read PC:PB	Stores as Bank.	
4.1		Inc. PC .	
4.2	Read Address:Bank	Store as Operand.	
5.1		Perform A &= Operand. Set N based off (A .L & \$80) and Z based off A .L.	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

BMI (30)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC . Decide to branch if N is 1.
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set PC .H to Address.H
4.2		Store as OpCode.
+X.1		

AND (31) Cycles: 5		Size: 2
Direct Indirect Indexed (d),y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

AND (32)		Cycles: 5	Size: 2
Direct Indirect (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Pointer: DB	Store as Operand.	
6.1		Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

AND (33)		Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB .	Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).	
2.2			
3.1			
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1		Perform Tmp = ui32(Pointer + Y).	
		Pointer = ui16(Tmp).	
		Bank = DB + (Tmp >> 16).	
5.2			
6.1			
6.2	Read Pointer:Bank	Store as Operand.	
7.1		Perform A &= Operand. Set N based off (A .L & \$80) and Z based off A .L.	
7.2	Read PC:PB .	Store as OpCode.	
+X.1			

BIT (34)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB .	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Read PC:PB .	Set Z based off (A.L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40).
5.2		Store as OpCode.
+X.1		

AND (35)		Cycles: 4	Size: 2
Direct, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Operand.	
2.1		Inc. PC .	
2.2			
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.	
3.2			
4.1			
4.2	Read Address	Store as Operand.	
5.1	Read PC:PB	Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .	
5.2		Store as OpCode.	
+X.1			

ROL (36)	Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Write to Address	Write Operand.L.
5.2		Set C based off (Operand.L & \$80). Perform Operand.L = (Operand.L << 1) C . Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.1		
6.2	Write to Address	Write Operand.L.
7.1	Read PC:PB	
7.2		Store as OpCode.
+X.1		

AND (37)		Cycles: 6	Size: 2
Direct Indirect Indexed Long [d],y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.	
5.1			
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.	
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).	
6.2	Read Pointer:Bank	Store as Operand.	
7.1		Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .	
7.2	Read PC:PB	Store as OpCode.	
+X.1			

SEC (38)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set the C flag to 1.
2.2	Read PC:PB	Store as OpCode.
+X.1		

AND (39)		Cycles: 4	Size: 3
Absolute, Y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer.	
		Tmp = ui32(Pointer + Y).	
		Pointer = ui16(Tmp).	
		If Orig.H == Pointer.H, skip 2 half-cycles.	
		Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand.	
5.1		Perform A &= Operand. Set N based off (A .L & \$80) and Z based off A .L.	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

DEC (3A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform A.L -= 1, set N based off (A.L & \$80), set Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

TSC (3B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set A to (S .L \$0100). Set N based off (A .H & \$80) and Z based off A .
2.2	Read PC:PB	Store as OpCode.
+X.1		

BIT (3C)	Cycles: 4	Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set Z based off (A .L & Operand.L), set N based off (Operand.L & \$80), and set V based off (Operand.L & \$40).
5.2	Read PC:PB	Store as OpCode.
+X.1		

AND (3D)		Cycles: 4	Size: 3
Absolute, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer.	
		Tmp = ui32(Pointer + X).	
		Pointer = ui16(Tmp).	
		If Orig.H == Pointer.H, skip 2 half-cycles.	
		Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand.	
5.1		Perform A &= Operand. Set N based off (A .L & \$80) and Z based off A .L.	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

ROL (3E)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB .	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\text{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set C based off (Operand.L & \$80). Perform $\text{Operand.L} = (\text{Operand.L} \ll 1) \mid \text{C}$. Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB .	Store as OpCode.
+X.1		

AND (3F)		Cycles: 5	Size: 4
Absolute Long, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB .	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB .	Store as Pointer.H.	
3.1		Inc. PC .	
3.2	Read PC:PB .	Stores as Bank.	
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$.	
		Pointer = $\text{ui16}(\text{Tmp})$.	
		Bank += (Tmp >> 16).	
4.2	Read Pointer:Bank	Store as Operand.	
5.1		Perform A &= Operand. Set N based off (A.L & \$80) and Z based off A.L .	
5.2	Read PC:PB .	Store as OpCode.	
+X.1			

RTI (40)	Cycles: 6	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S.L by 1 and set S.H to 1.
3.2	Read ((S.L \$0100)+0)	Store as Operand.L.
4.1		Set P to ((Operand.L & ~\$30) (P & \$30)).
4.2	Read ((S.L \$0100)+1)	Store as Operand.L.
5.1		
5.2	Read ((S.L \$0100)+2)	Store as Operand.H.
6.1		Inc. S.L by 2 and set S.H to 1. Set PC to Operand.
6.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (41)	Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address.L = Pointer.L. Address.H = D .H.
2.2		If D .L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer: DB	Store as Operand.
7.1		Perform A ^= Operand. Set N based off (A .L & \$80) and Z based off A .L.
7.2	Read PC:PB	Store as OpCode.
+X.1		

WDM (42) Cycles: 2		Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (43)	Cycles: 4	Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Perform A ^= Operand. Set N based off (A .L & \$80) and Z based off A .L.
4.2	Read PC:PB	Store as OpCode.
+X.1		

MVP (44)	Cycles: 7	Size: 3
Block Move (Positive)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set DB to Operand.L.
2.2	Read PC:PB	Stores as Bank.
3.1		Inc. PC .
3.2	Read X:Bank	Store as Operand.L.
4.1		
4.2	Write Y:DB	Write Operand.L.
5.1		Perform A -= 1. If X flag is set, perform X.L -= 1 and Y.L -= 1, otherwise perform X -= 1 and Y -= 1. If A is not \$FFFF, perform PC -= 3.
5.2		
6.1		
6.2		
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (45)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

LSR (46)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set C based off (Operand.L & \$01). Perform Operand.L >>= 1. Set N based off (Operand.L & \$80) and Z based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (47)	Cycles: 6	Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

PHA (48)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Write to ((S.L \$0100)+0)	Inc. PC .
1.2		
2.1		
2.2		Push A.L onto stack.
3.1		Dec. S.L by 1 and set S.H to 1.
3.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (49)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

LSR (4A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set C based off (A.L & \$01). Perform A.L >>= 1. Set N based off (A.L & \$80) and Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

PHK (4B)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Write to ((S.L \$0100)+0)	Inc. PC .
1.2		
2.1		
2.2		Push PB onto stack.
3.1		Dec. S.L by 1 and set S.H to 1.
3.2	Read PC:PB	Store as OpCode.
+X.1		

JMP (4C)	Cycles: 3	Size: 3
Absolute		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Set PC to Address.
3.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (4D)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Perform A ^= Operand. Set N based off (A .L & \$80) and Z based off A .L.
4.2	Read PC:PB	Store as OpCode.
+X.1		

LSR (4E)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		
4.2	Write to Address: DB	Write Operand.L.
5.1		Set C based off (Operand.L & \$01). Perform Operand.L >>= 1. Set N based off (Operand.L & \$80) and Z based off Operand.L.
5.2	Write to Address: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (4F)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.L.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand.
5.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

BVC (50)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC . Decide to branch if V is 0.
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set PC .H to Address.H
4.2		Store as OpCode.
+X.1		

EOR (51)	Cycles: 5	Size: 2
Direct Indirect Indexed (d),y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (52)	Cycles: 5	Size: 2
Direct Indirect (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer: DB	Store as Operand.
6.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
6.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (53)	Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform A ^= Operand. Set N based off (A .L & \$80) and Z based off A .L.
7.2	Read PC:PB	Store as OpCode.
+X.1		

MVN (54)	Cycles: 7	Size: 3
Block Move (Negative)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set DB to Operand.L.
2.2	Read PC:PB	Stores as Bank.
3.1		Inc. PC .
3.2	Read X:Bank	Store as Operand.L.
4.1		
4.2	Write Y:DB	Write Operand.L.
5.1		Perform A -= 1. If X flag is set, perform X.L += 1 and Y.L += 1, otherwise perform X += 1 and Y += 1. If A is not \$FFFF, perform PC -= 3.
5.2		
6.1		
6.2		
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (55)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Read PC:PB	Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2		Store as OpCode.
+X.1		

LSR (56)	Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Write to Address	Write Operand.L.
5.2		Set C based off (Operand.L & \$01). Perform Operand.L >>= 1. Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.1		
6.2	Write to Address	Write Operand.L.
7.1	Read PC:PB	
7.2		Store as OpCode.
+X.1		

EOR (57)		Cycles: 6	Size: 2
Direct Indirect Indexed Long [d],y (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.	
5.1			
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.	
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).	
6.2	Read Pointer:Bank	Store as Operand.	
7.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .	
7.2	Read PC:PB	Store as OpCode.	
+X.1			

CLI (58)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set the I flag to 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (59)	Cycles: 4	Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

PHY (5A)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Write to ((S.L \$0100)+0)	Inc. PC .
1.2		
2.1		
2.2		Push Y.L onto stack.
3.1		Dec. S.L by 1 and set S.H to 1.
3.2	Read PC:PB	Store as OpCode.
+X.1		

TCD (5B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		Set D to A . Set N based off (D .H & \$80) and Z based off D .
2.2		Store as OpCode.
+X.1		

JML (5C)	Cycles: 4	Size: 4
Absolute Long		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Copy Address to PC . Copy Bank to PB .
4.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (5D) Cycles: 4		Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB .	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB .	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform A ^= Operand. Set N based off (A.L & \$80) and Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

LSR (5E)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set C based off (Operand.L & \$01). Perform $\text{Operand.L} \gg= 1$. Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

EOR (5F)		Cycles: 5	Size: 4
Absolute Long, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC .	
3.2	Read PC:PB	Stores as Bank.	
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$.	
		Pointer = $\text{ui16}(\text{Tmp})$.	
		Bank += (Tmp >> 16).	
4.2	Read Pointer:Bank	Store as Operand.	
5.1		Perform $\text{A} \wedge = \text{Operand}$. Set N based off (A.L & \$80) and Z based off A.L .	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

RTS (60)	Cycles: 6	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read ((S.L \$0100)+0)	Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S.L by 1 and set S.H to 1.
3.2		Store as Operand.L.
4.1		
4.2		Store as Operand.H.
5.1		
5.2		
6.1		Inc. S.L by 1 and set S.H to 1. Set PC to (Operand + 1).
6.2		Store as OpCode.
+X.1		

ADC (61)	Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address.L = Pointer.L. Address.H = D .H.
2.2		If D .L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1	Read Address	Store as Pointer.
4.2		
5.1	Read Address + 1	Store as Pointer.H.
5.2		
6.1	Read Pointer: DB	Store as Operand.
6.2		
7.1		If D flag == 0: Result = ui16(A .L) + ui16(Operand) + C . V flag = (~ui16(A .L) ^ ui16(Operand)) & (ui16(A .L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A .L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A .L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A .L >> 4) ^ (Operand >> 4)) & ((A .L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2		Store as OpCode.
+X.1		

PER (62)	Cycles: 6	Size: 3
Program Counter Relative Long		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		Inc. PC . Set Operand to Operand + PC .
3.2		
4.1		
4.2	Write to ((S .L \$0100)+0)	Push Operand.H onto stack.
5.1		
5.2	Write to ((S .L \$0100)-1)	Push Operand.L onto stack.
6.1		Dec. S .L by 2 and set S .H to 1.
6.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (63)		Cycles: 4	Size: 2
Stack Relative (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).	
2.2			
3.1			
3.2	Read Address	Store as Operand.	
4.1		If D flag == 0: Result = ui16(A .L) + ui16(Operand) + C . V flag = (~(ui16(A .L) ^ ui16(Operand)) & (ui16(A .L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A .L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A .L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A .L >> 4) ^ (Operand >> 4)) & ((A .L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
4.2		Store as OpCode.	
+X.1			

STZ (64)	Cycles: 3	Size: 2
Direct (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Write to Address	Write 0.
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (65)		Cycles: 3	Size: 2
Direct (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand.	
4.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.	
4.2	Read PC:PB	Store as OpCode.	
+X.1			

ROR (66)		Cycles: 5	Size: 2
Direct (Read/Modify/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Operand.	
4.1			
4.2	Write to Address	Write Operand.L.	
5.1		Set C based off (Operand.L & \$01). Perform Operand.L = (Operand.L >> 1) (C << 7). Set N based off (Operand.L & \$80) and Z based off Operand.L.	
5.2	Write to Address	Write Operand.L.	
6.1			
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (67) Cycles: 6		Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: $Result = ui16(A.L) + ui16(Operand) + C.$ $V\text{ flag} = (\sim(ui16(A.L) \wedge ui16(Operand)) \& (ui16(A.L) \wedge ui8(Result)) \& \$80) \neq 0.$ $C\text{ flag} = Result > \$FF.$ $Result = ui8(Result). Z\text{ flag} = Result == \$00. N\text{ flag} = (Result \& \$80) \neq 0.$ If D flag == 1: $Lo = ui16(A.L \& 0x0F) + ui16(Operand \& 0x0F) + C;$ if $(Lo > 9) Lo += 6.$ $CarryToHi = Lo > \$0F ? 1 : 0.$ $HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi.$ $V\text{ flag} = (\sim((A.L >> 4) \wedge (Operand >> 4)) \& ((A.L >> 4) \wedge HiSum) \& \$08) \neq 0.$ if $(HiSum > 9) HiSum += 6.$ $C\text{ flag} = HiSum > \$0F.$ $Result = ui8(((HiSum \& 0x0F) << 4) (Lo \& \$0F)).$ $Z\text{ flag} = Result == \$00. N\text{ flag} = (Result \& \$80) \neq 0.$
7.2	Read PC:PB	Store as OpCode.
+X.1		

PLA (68)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S.L by 1 and set S.H to 1.
3.2		Store as Operand.L.
4.1		Set A.L to Operand.L. Set N based off (A.L & \$80) and Z based off A.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (69)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = ~(A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (6A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set C based off (A.L & \$01). Perform A.L = (A.L >> 1) (C << 7). Set N based off (A.L & \$80) and Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

RTL (6B)	Cycles: 6	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		
3.2	Read u8(S .L+1) \$0100	Store as Operand.L.
4.1		
4.2	Read u8(S .L+2) \$0100	Store as Operand.H.
5.1		
5.2	Read ((S .L \$0100)+3)	Store as PB .
6.1		Inc. S .L by 3 and set S .H to 1. Set PC to (Operand + 1). Set PB to Bank.
6.2	Read PC:PB	Store as OpCode.
+X.1		

JMP (6C)		Cycles: 5	Size: 3
Absolute Indirect (Jump)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC .	
3.2	Read Pointer	Store as Address.	
4.1			
4.2	Read Pointer + 1	Store as Address.H.	
5.1		Set PC to Address.	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (6D) Cycles: 4		Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (6E)		Cycles: 6	Size: 3
Absolute (Read/Modify/Write)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Address.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Address.H.	
3.1		Inc. PC .	
3.2	Read Address: DB	Store as Operand.	
4.1			
4.2	Write to Address: DB	Write Operand.L.	
5.1		Set C based off (Operand.L & \$01). Perform Operand.L = (Operand.L >> 1) (C << 7). Set N based off (Operand.L & \$80) and Z based off Operand.L.	
5.2	Write to Address: DB	Write Operand.L.	
6.1			
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (6F)		Cycles: 5	Size: 4
Absolute Long (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Address.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Address.H.	
3.1		Inc. PC .	
3.2	Read PC:PB	Stores as Bank.	
4.1		Inc. PC .	
4.2	Read Address:Bank	Store as Operand.	
5.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~(ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~(A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

BVS (70)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC . Decide to branch if V is 1.
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set PC .H to Address.H
4.2		Store as OpCode.
+X.1		

ADC (71) Cycles: 5		Size: 2
Direct Indirect Indexed (d),y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (72)		Cycles: 5	Size: 2
Direct Indirect (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Pointer: DB	Store as Operand.	
6.1		If D flag == 0: $Result = ui16(A.L) + ui16(Operand) + C.$ $V\text{ flag} = (\sim(ui16(A.L) \wedge ui16(Operand)) \wedge (ui16(A.L) \wedge ui8(Result)) \& \$80) \neq 0.$ $C\text{ flag} = Result > \$FF.$ $Result = ui8(Result). Z\text{ flag} = Result == \$00. N\text{ flag} = (Result \& \$80) \neq 0.$ If D flag == 1: $Lo = ui16(A.L \& 0x0F) + ui16(Operand \& 0x0F) + C;$ if $(Lo > 9)$ $Lo += 6.$ $CarryToHi = Lo > \$0F ? 1 : 0.$ $HiSum = ui16(A.L \gg 4) + ui16(Operand \gg 4) + CarryToHi.$ $V\text{ flag} = (\sim((A.L \gg 4) \wedge (Operand \gg 4)) \wedge ((A.L \gg 4) \wedge HiSum) \& \$08) \neq 0.$ if $(HiSum > 9)$ $HiSum += 6.$ $C\text{ flag} = HiSum > \$0F.$ $Result = ui8(((HiSum \& 0x0F) << 4) (Lo \& \$0F)).$ $Z\text{ flag} = Result == \$00. N\text{ flag} = (Result \& \$80) \neq 0.$	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (73) Cycles: 7		Size: 2
Stack Relative Indirect Indexed (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S.L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

STZ (74)	Cycles: 4	Size: 2
Direct, X (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2	Write to Address	
4.1		
4.2		Write 0.
5.1	Read PC:PB	
5.2		Store as OpCode.
+X.1		

ADC (75) Cycles: 4		Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (76)	Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Set C based off (Operand.L & \$01). Perform Operand.L = (Operand.L >> 1) (C << 7). Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (77) Cycles: 6		Size: 2
Direct Indirect Indexed Long [d],y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

SEI (78)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set the I flag to 1.
2.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (79) Cycles: 4		Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

PLY (7A)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S.L by 1 and set S.H to 1.
3.2		Store as Operand.L.
4.1	Set Y.L to Operand.L. Set N based off (Y.L & \$80). Set Z based off Y.L .	
4.2	Read PC:PB	Store as OpCode.
+X.1		

TDC (7B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set A to D . Set N based off (A .H & \$80) and Z based off A .
2.2	Read PC:PB	Store as OpCode.
+X.1		

JMP (7C)		Cycles: 6	Size: 3
Absolute Indexed Indirect (a,x) (Jump			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Perform Pointer += X . Bank = PB .	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Address.L.	
5.1			
5.2	Read Pointer + 1:Bank	Store as Address.H.	
6.1		Set PC to Address.	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

ADC (7D) Cycles: 4		Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC. Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		If D flag == 0: Result = ui16(A.L) + ui16(Operand) + C . V flag = (~ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80 != 0. C flag = Result > \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = ui16(A.L & 0x0F) + ui16(Operand & 0x0F) + C ; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = ui16(A.L >> 4) + ui16(Operand >> 4) + CarryToHi. V flag = (~((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

ROR (7E)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\text{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set C based off (Operand.L & \$01). Perform $\text{Operand.L} = (\text{Operand.L} \gg 1) \mid (\text{C} \ll 7)$. Set N based off (Operand.L & \$80) and Z based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

ADC (7F)		Cycles: 5	Size: 4
Absolute Long, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1	Read PC:PB	Inc. PC .	
2.2		Store as Pointer.H.	
3.1	Read PC:PB	Inc. PC .	
3.2		Stores as Bank.	
4.1	Read Pointer:Bank	Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$.	
4.2		Pointer = $\text{ui16}(\text{Tmp})$. Bank += $(\text{Tmp} \gg 16)$.	
5.1	Read PC:PB	Store as Operand.	
5.2		If D flag == 0: Result = $\text{ui16}(\text{A.L}) + \text{ui16}(\text{Operand}) + \text{C}$. V flag = $(\sim(\text{ui16}(\text{A.L}) \wedge \text{ui16}(\text{Operand})) \& (\text{ui16}(\text{A.L}) \wedge \text{ui8}(\text{Result})) \& \$80) \neq 0$. C flag = Result > \$FF. Result = $\text{ui8}(\text{Result})$. Z flag = Result == \$00. N flag = $(\text{Result} \& \$80) \neq 0$. If D flag == 1: Lo = $\text{ui16}(\text{A.L} \& 0x0F) + \text{ui16}(\text{Operand} \& 0x0F) + \text{C}$; if (Lo > 9) Lo += 6. CarryToHi = Lo > \$0F ? 1 : 0. HiSum = $\text{ui16}(\text{A.L} \gg 4) + \text{ui16}(\text{Operand} \gg 4) + \text{CarryToHi}$. V flag = $(\sim((\text{A.L} \gg 4) \wedge (\text{Operand} \gg 4)) \& ((\text{A.L} \gg 4) \wedge \text{HiSum}) \& \$08) \neq 0$. if (HiSum > 9) HiSum += 6. C flag = HiSum > \$0F. Result = $\text{ui8}(((\text{HiSum} \& 0x0F) \ll 4) \mid (\text{Lo} \& \$0F))$. Z flag = Result == \$00. N flag = $(\text{Result} \& \$80) \neq 0$.	
+X.1		Store as OpCode.	

BRA (80)	Cycles: 3	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC . Decide to branch always.
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set PC .H to Address.H
4.2		Store as OpCode.
+X.1		

STA (81)	Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address.L = Pointer.L. Address.H = D .H.
2.2		If D .L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		Set Operand.L to A .L.
6.2	Write to Pointer: DB	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

BRL (82)	Cycles: 4	Size: 3
Program Counter Relative Long		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		Inc. PC .
3.2		
4.1		Perform PC += Operand.
4.2	Read PC:PB	Store as OpCode.
+X.1		

STA (83)	Cycles: 4	Size: 2
Stack Relative (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		Set Operand to A .
3.2	Write to Address	Write Operand.L.
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STY (84)	Cycles: 3	Size: 2
Direct (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Write to Address	Write Y.L .
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STA (85)	Cycles: 3	Size: 2
Direct (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Write to Address	Write A.L .
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STX (86)	Cycles: 3	Size: 2
Direct (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Write to Address	Write X.L .
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STA (87)	Cycles: 6	Size: 2
Direct Indirect Long (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		Set Operand to A .
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

DEY (88)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform Y.L -= 1, set N based off (Y.L & \$80), set Z based off Y.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

BIT (89)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set Z based off (A.L & Operand.L).
2.2	Read PC:PB	Store as OpCode.
+X.1		

TXA (8A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform A.L = X.L , set N based off (A.L & \$80), set Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

PHB (8B)		Cycles: 3	Size: 1
Implied			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1			Inc. PC .
1.2			
2.1			
2.2	Write to ((S .L \$0100)+0)		Push DB onto stack.
3.1			Dec. S .L by 1 and set S .H to 1.
3.2	Read PC:PB		Store as OpCode.
+X.1			

STY (8C)	Cycles: 4	Size: 3
Absolute (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc PC . Set Operand to Y .
3.2	Write to Address: DB	Write Operand.L.
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STA (8D)	Cycles: 4	Size: 3
Absolute (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc PC . Set Operand to A .
3.2	Write to Address: DB	Write Operand.L.
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STX (8E)	Cycles: 4	Size: 3
Absolute (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc PC . Set Operand to X .
3.2	Write to Address: DB	Write Operand.L.
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STA (8F)	Cycles: 5	Size: 4
Absolute Long (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc PC . Set Operand to A .
4.2	Write to Address:Bank	Write Operand.L.
5.1		
5.2	Read PC:PB	Store as OpCode.
+X.1		

BCC (90)		Cycles: 2	Size: 2
Relative			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC . Decide to branch if C is 0.	
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.	
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).	
2.2			
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).	
3.2		Poll interrupts.	
4.1		Set PC .H to Address.H	
4.2		Store as OpCode.	
+X.1			

STA (91)	Cycles: 6	Size: 2
Direct Indirect Indexed (d),y (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{Y})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\text{DB} + (\text{Tmp} \gg 16)$.
5.2		
6.1		Set Operand to A .
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

STA (92)	Cycles: 5	Size: 2
Direct Indirect (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Set Operand to A .
5.2	Write to Pointer: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

STA (93)	Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank = DB + (Tmp >> 16).
5.2		
6.1		Set Operand to A .
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

STY (94)	Cycles: 4	Size: 2
Direct, X (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write Y.L .
5.1	Read PC:PB	
5.2		Store as OpCode.
+X.1		

STA (95)	Cycles: 4	Size: 2
Direct, X (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write A.L .
5.1	Read PC:PB	Store as OpCode.
5.2		
+X.1		

STX (96)	Cycles: 4	Size: 2
Direct, Y (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + Y.L + D.L , Address.H = D.H , otherwise Address = Operand + Y + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write X.L .
5.1		
5.2	Read PC:PB	Store as OpCode.
+X.1		

STA (97)	Cycles: 6	Size: 2
Direct Indirect Indexed Long [d],y (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Write to Pointer:Bank	Write A.L .
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

TYA (98)		Cycles: 2	Size: 1
Implied			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1			Inc. PC .
1.2	Read PC:PB		Discard.
2.1			Perform A.L = Y.L , set N based off (A.L & \$80), set Z based off A.L .
2.2	Read PC:PB		Store as OpCode.
+X.1			

STA (99)	Cycles: 5	Size: 3
Absolute, Y (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{Y})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		Set Operand to A .
4.2	Write to Pointer:Bank	Write Operand.L.
5.1		
5.2	Read PC:PB	Store as OpCode.
+X.1		

TXS (9A)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set S.L to X.L and S.H to 1
2.2	Read PC:PB	Store as OpCode.
+X.1		

TXY (9B)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform Y.L = X.L , set N based off (Y.L & \$80), set Z based off Y.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

STZ (9C)	Cycles: 4	Size: 3
Absolute (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc PC . Set Operand to 0.
3.2	Write to Address: DB	Write Operand.L.
4.1		
4.2	Read PC:PB	Store as OpCode.
+X.1		

STA (9D)	Cycles: 5	Size: 3
Absolute, X (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		Set Operand to A .
4.2	Write to Pointer:Bank	Write Operand.L.
5.1		
5.2	Read PC:PB	Store as OpCode.
+X.1		

STZ (9E)	Cycles: 5	Size: 3
Absolute, X (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		Set Operand to 0.
4.2	Write to Pointer:Bank	Write Operand.L.
5.1		
5.2	Read PC:PB	Store as OpCode.
+X.1		

STA (9F)	Cycles: 5	Size: 4
Absolute Long, X (Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank += $(\text{Tmp} \gg 16)$.
4.2	Write to Pointer:Bank	Write A.L .
5.1		
5.2	Read PC:PB	Store as OpCode.
+X.1		

LDY (A0)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set Y.L to Operand.L. Set N based off (Y.L & \$80). Set Z based off Y.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (A1)	Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address.L = Pointer.L. Address.H = D .H.
2.2		If D .L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer: DB	Store as Operand.
7.1		Set A .L to Operand.L. Set N based off (A .L & \$80). Set Z based off A .L.
7.2	Read PC:PB	Store as OpCode.
+X.1		

LDX (A2)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set X.L to Operand.L. Set N based off (X.L & \$80). Set Z based off X.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (A3)	Cycles: 4	Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1	Read PC:PB	Set A .L to Operand.L. Set N based off (A .L & \$80). Set Z based off A .L.
4.2		Store as OpCode.
+X.1		

LDY (A4)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Set Y.L to Operand.L. Set N based off (Y.L & \$80). Set Z based off Y.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (A5)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

LDX (A6)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Set X.L to Operand.L. Set N based off (X.L & \$80). Set Z based off X.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (A7)	Cycles: 6	Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

TAY (A8)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform Y.L = A.L , set N based off (Y.L & \$80), set Z based off Y.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (A9)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

TAX (AA)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform X.L = A.L , set N based off (X.L & \$80), set Z based off X.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

PLB (AB)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read ((S.L \$0100)+0)	Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S.L by 1 and set S.H to 1.
3.2		Store as Bank.
4.1		Set DB to Bank, set N based off (DB & \$80), and set Z based off DB .
4.2		Store as OpCode.
+X.1		

LDY (AC)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Set Y .L to Operand.L. Set N based off (Y .L & \$80). Set Z based off Y .L.
4.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (AD)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

LDX (AE)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Set X.L to Operand.L. Set N based off (X.L & \$80). Set Z based off X.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (AF)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand.
5.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

BCS (B0)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC . Decide to branch if C is 1.
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set PC .H to Address.H
4.2		Store as OpCode.
+X.1		

LDA (B1)	Cycles: 5	Size: 2
Direct Indirect Indexed (d),y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (B2)	Cycles: 5	Size: 2
Direct Indirect (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer: DB	Store as Operand.
6.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
6.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (B3)	Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set A .L to Operand.L. Set N based off (A .L & \$80). Set Z based off A .L.
7.2	Read PC:PB	Store as OpCode.
+X.1		

LDY (B4)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Read PC:PB	Set Y.L to Operand.L. Set N based off (Y.L & \$80). Set Z based off Y.L .
5.2		Store as OpCode.
+X.1		

LDA (B5)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Read PC:PB	Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
5.2		Store as OpCode.
+X.1		

LDX (B6)	Cycles: 4	Size: 2
Direct, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + Y.L + D.L , Address.H = D.H , otherwise Address = Operand + Y + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1	Read PC:PB	Set X.L to Operand.L. Set N based off (X.L & \$80). Set Z based off X.L .
5.2		Store as OpCode.
+X.1		

LDA (B7) Cycles: 6		Size: 2
Direct Indirect Indexed Long [d],y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
7.2	Read PC:PB	Store as OpCode.
+X.1		

CLV (B8)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set the V flag to 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (B9)	Cycles: 4	Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

TSX (BA)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform X.L = S.L , set N based off (X.L & \$80), set Z based off X.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

TYX (BB)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform X.L = Y.L , set N based off (X.L & \$80), set Z based off X.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

LDY (BC)	Cycles: 4	Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set Y .L to Operand.L. Set N based off (Y .L & \$80). Set Z based off Y .L.
5.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (BD)	Cycles: 4	Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

LDX (BE)	Cycles: 4	Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set X.L to Operand.L. Set N based off (X.L & \$80). Set Z based off X.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

LDA (BF)	Cycles: 5	Size: 4
Absolute Long, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set A.L to Operand.L. Set N based off (A.L & \$80). Set Z based off A.L .
5.2	Read PC:PB	Store as OpCode.
+X.1		

CPY (C0)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC . Set C flag based off (Y .L >= Operand.L), set Z flag based off (Y .L == Operand.L), and set N flag based off ((Y .L - Operand.L) & \$80) != 0.
2.2		Store as OpCode.
+X.1		

CMP (C1)		Cycles: 6	Size: 2
Direct Indexed Indirect (d,x) (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Pointer.	
2.1		Inc. PC . Set Pointer to Pointer + X + D .	
2.2		Address.L = Pointer.L. Address.H = D .H.	
3.1		If D .L is 0, skip the next cycle.	
3.2		Address.H = Pointer.H (fixes high byte of address).	
4.1	Read Address		
4.2		Store as Pointer.	
5.1	Read Address + 1		
5.2		Store as Pointer.H.	
6.1	Read Pointer: DB		
6.2		Store as Operand.	
7.1		Set C flag based off (A .L >= Operand.L), set Z flag based off (A .L == Operand.L), and set N flag based off ((A .L - Operand.L) & \$80) != 0.	
7.2	Read PC:PB	Store as OpCode.	
+X.1			

REP (C2)	Cycles: 3	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		P &= ~(Operand.L & ~(X flag M flag)).
3.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (C3)	Cycles: 4	Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Set C flag based off (A .L >= Operand.L), set Z flag based off (A .L == Operand.L), and set N flag based off ((A .L - Operand.L) & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

CPY (C4)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Set C flag based off (Y.L >= Operand.L), set Z flag based off (Y.L == Operand.L), and set N flag based off ((Y.L - Operand.L) & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (C5)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

DEC (C6)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Perform Operand.L -= 1, set N based off (Operand.L & \$80) and Z based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (C7)		Cycles: 6	Size: 2
Direct Indirect Long (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Address + 2	Store as Bank.	
6.1			
6.2	Read Pointer:Bank	Store as Operand.	
7.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.	
7.2	Read PC:PB	Store as OpCode.	
+X.1			

INY (C8)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform Y.L += 1, set N based off (Y.L & \$80), set Z based off Y.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (C9)		Cycles: 2	Size: 2
Immediate			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1			Inc. PC .
1.2	Read PC:PB		Store as Operand.
2.1			Inc. PC . Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.
2.2	Read PC:PB		Store as OpCode.
+X.1			

DEX (CA)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform X.L -= 1, set N based off (X.L & \$80), set Z based off X.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

WAI (CB)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		
2.2		
3.1		
3.2		Enter waiting state. Repeat this cycle forever.

CPY (CC)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Set C flag based off (Y .L >= Operand.L), set Z flag based off (Y .L == Operand.L), and set N flag based off ((Y .L - Operand.L) & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (CD)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

DEC (CE)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		
4.2	Write to Address: DB	Write Operand.L.
5.1		Perform Operand.L -= 1, set N based off (Operand.L & \$80) and Z based off Operand.L.
5.2	Write to Address: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (CF)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand.
5.1		Set C flag based off (A .L >= Operand.L), set Z flag based off (A .L == Operand.L), and set N flag based off ((A .L - Operand.L) & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

BNE (D0)	Cycles: 2	Size: 2
Relative		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC . Decide to branch if Z is 0.
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set PC .H to Address.H
4.2		Store as OpCode.
+X.1		

CMP (D1) Cycles: 5		Size: 2
Direct Indirect Indexed (d),y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (D2)		Cycles: 5	Size: 2
Direct Indirect (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.	
2.1		Inc. PC .	
2.2			
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .	
3.2	Read Address	Store as Pointer.	
4.1			
4.2	Read Address + 1	Store as Pointer.H.	
5.1			
5.2	Read Pointer: DB	Store as Operand.	
6.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.	
6.2	Read PC:PB	Store as OpCode.	
+X.1			

CMP (D3)	Cycles: 7	Size: 2
Stack Relative Indirect Indexed (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set C flag based off (A .L >= Operand.L), set Z flag based off (A .L == Operand.L), and set N flag based off ((A .L - Operand.L) & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

PEI (D4)	Cycles: 6	Size: 2
Direct Page Indirect		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		
4.2	If D.L is 0, Read (((Address.L + 1) & \$FF) (Address.H << 8)), otherwise read Address + 1	Store as Bank.
5.1		
5.2	Write to ((S.L \$0100)+0)	Push Operand.H onto stack.
6.1		
6.2	Write to ((S.L \$0100)-1)	Push Operand.L onto stack.
7.1		Dec. S.L by 2 and set S.H to 1.
7.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (D5)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2		Store as Operand.
5.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.
5.2		Store as OpCode.
+X.1		

DEC (D6)	Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Perform Operand.L -= 1, set N based off (Operand.L & \$80), and Z based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (D7) Cycles: 6		Size: 2
Direct Indirect Indexed Long [d],y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off ((A.L - Operand.L) & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

CLD (D8)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set the D flag to 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (D9) Cycles: 4		Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set C flag based off (A .L >= Operand.L), set Z flag based off (A .L == Operand.L), and set N flag based off ((A .L - Operand.L) & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

PHX (DA)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Write to ((S.L \$0100)+0)	Inc. PC .
1.2		
2.1		
2.2		Push X.L onto stack.
3.1		Dec. S.L by 1 and set S.H to 1.
3.2	Read PC:PB	Store as OpCode.
+X.1		

STP (DB)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		
3.2		
4.1		Stop processor.

JML (DC)	Cycles: 6	Size: 3
Absolute Indirect Long (Jump)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC .
3.2	Read Pointer	Store as Operand.
4.1		
4.2	Read Pointer + 1	Store as Operand.H.
5.1		
5.2	If D.L is 0, Read (((Pointer.L + 2) & \$FF) (Pointer.H << 8)), otherwise read Pointer + 2	Store as Bank.
6.1		Set PC to Operand. Set PB to Bank.
6.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (DD)		Cycles: 4	Size: 3
Absolute, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC . Perform Orig = Pointer.	
		Tmp = ui32(Pointer + X).	
		Pointer = ui16(Tmp).	
		If Orig.H == Pointer.H, skip 2 half-cycles.	
		Bank = DB + (Tmp >> 16).	
3.2			
4.1			
4.2	Read Pointer:Bank	Store as Operand.	
5.1		Set C flag based off (A .L >= Operand.L), set Z flag based off (A .L == Operand.L), and set N flag based off ((A .L - Operand.L) & \$80) != 0.	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

DEC (DE)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Perform $\text{Operand.L} -= 1$, set N based off ($\text{Operand.L} \& \$80$), and Z based off Operand.L .
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

CMP (DF)		Cycles: 5	Size: 4
Absolute Long, X (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1		Inc. PC .	
1.2	Read PC:PB	Store as Pointer.	
2.1		Inc. PC .	
2.2	Read PC:PB	Store as Pointer.H.	
3.1		Inc. PC .	
3.2	Read PC:PB	Stores as Bank.	
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank += $(\text{Tmp} \gg 16)$.	
4.2	Read Pointer:Bank	Store as Operand.	
5.1		Set C flag based off (A.L >= Operand.L), set Z flag based off (A.L == Operand.L), and set N flag based off $((\mathbf{A.L} - \text{Operand.L}) \& \$80) \neq 0$.	
5.2	Read PC:PB	Store as OpCode.	
+X.1			

CPX (E0)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC . Set C flag based off (X .L >= Operand.L), set Z flag based off (X .L == Operand.L), and set N flag based off ((X .L - Operand.L) & \$80) != 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (E1) Cycles: 6		Size: 2
Direct Indexed Indirect (d,x) (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Pointer to Pointer + X + D . Address.L = Pointer.L. Address.H = D .H.
2.2		If D .L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer: DB	Store as Operand.
7.1		If D flag == 0: Result = ui16(A .L) - ui16(Operand) - (1 - C). V flag = ((ui16(A .L) ^ ui16(Operand)) & (ui16(A .L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A .L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A .L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A .L >> 4) ^ (Operand >> 4)) & ((A .L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

SEP (E2)	Cycles: 3	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		
2.2		
3.1		Perform P = (Operand.L & ~(X flag M flag)).
3.2		Store as OpCode.
+X.1		

SBC (E3)	Cycles: 4	Size: 2
Stack Relative (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		If D flag == 0: Result = ui16(A .L) - ui16(Operand) - (1 - C). V flag = ((ui16(A .L) ^ ui16(Operand)) & (ui16(A .L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A .L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A .L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A .L >> 4) ^ (Operand >> 4)) & ((A .L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
4.2		Read PC:PB
		Store as OpCode.
+X.1		

CPX (E4)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		Set C flag based off (X.L >= Operand.L), set Z flag based off (X.L == Operand.L), and set N flag based off ((X.L - Operand.L) & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (E5)	Cycles: 3	Size: 2
Direct (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

INC (E6)	Cycles: 5	Size: 2
Direct (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Perform Operand.L += 1, set N based off (Operand.L & \$80), set Z based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (E7) Cycles: 6		Size: 2
Direct Indirect Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: $Result = ui16(A.L) - ui16(Operand) - (1 - C).$ $V\text{ flag} = ((ui16(A.L) \wedge ui16(Operand)) \& (ui16(A.L) \wedge ui8(Result)) \& \$80) \neq 0.$ $C\text{ flag} = Result \leq \$FF.$ $Result = ui8(Result).$ $Z\text{ flag} = Result == \$00.$ $N\text{ flag} = (Result \& \$80) \neq 0.$ If D flag == 1: $Lo = int16(A.L \& 0x0F) - int16(Operand \& 0x0F) - (1 - C);$ if $(Lo < 0)$ $Lo \text{ -= } 6.$ $BorrowToHi = Lo < 0 ? 1 : 0.$ $HiSum = int16(A.L \gg 4) - int16(Operand \gg 4) - BorrowToHi.$ $V\text{ flag} = (((A.L \gg 4) \wedge (Operand \gg 4)) \& ((A.L \gg 4) \wedge HiSum) \& \$08) \neq 0.$ if $(HiSum < 0)$ $HiSum \text{ -= } 6.$ $C\text{ flag} = HiSum \geq 0.$ $Result = ui8(((HiSum \& 0x0F) \ll 4) (Lo \& \$0F)).$ $Z\text{ flag} = Result == \$00.$ $N\text{ flag} = (Result \& \$80) \neq 0.$
7.2	Read PC:PB	Store as OpCode.
+X.1		

INX (E8)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Perform X.L += 1, set N based off (X.L & \$80), set Z based off X.L .
2.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (E9)	Cycles: 2	Size: 2
Immediate		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC.
1.2	Read PC:PB	Store as Operand.
2.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
2.2	Read PC:PB	Store as OpCode.
+X.1		

NOP (EA)		Cycles: 2	Size: 1
Implied			
Cycle	R/W		Desc.
-X.2	Read PC:PB		Store as OpCode.
1.1			Inc. PC .
1.2	Read PC:PB		Discard.
2.1			
2.2	Read PC:PB		Store as OpCode.
+X.1			

XBA (EB)	Cycles: 3	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2		
2.1		
2.2		
3.1		Swap A.L and A.H , set N based off (A.L & \$80), and set Z based off A.L .
3.2	Read PC:PB	Store as OpCode.
+X.1		

CPX (EC)	Cycles: 4	Size: 3
Absolute (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		Set C flag based off (X .L >= Operand.L), set Z flag based off (X .L == Operand.L), and set N flag based off ((X .L - Operand.L) & \$80) != 0.
4.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (ED)		Cycles: 4	Size: 3
Absolute (Read)			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC .	
1.2		Store as Address.	
2.1		Inc. PC .	
2.2		Store as Address.H.	
3.1		Inc. PC .	
3.2	Read Address: DB	Store as Operand.	
4.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.	
4.2		Store as OpCode.	
+X.1			

INC (EE)	Cycles: 6	Size: 3
Absolute (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read Address: DB	Store as Operand.
4.1		
4.2	Write to Address: DB	Write Operand.L.
5.1		Perform Operand.L += 1, set N based off (Operand.L & \$80), set Z based off Operand.L.
5.2	Write to Address: DB	Write Operand.L.
6.1		
6.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (EF)	Cycles: 5	Size: 4
Absolute Long (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Address.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Address.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC .
4.2	Read Address:Bank	Store as Operand.
5.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

BEQ (F0)		Cycles: 2	Size: 2
Relative			
Cycle	R/W	Desc.	
-X.2	Read PC:PB	Store as OpCode.	
1.1	Read PC:PB	Inc. PC . Decide to branch if Z is 1.	
1.2		Store as Operand. Address = i16(i8(Operand.L)) + PC , BoundaryCrossed = Address.H != PC .H. If not branching or not BoundaryCrossed, poll interrupts.	
2.1		Inc. PC . If not branching, end (next half-cycle is 4.2).	
2.2			
3.1		Set PC .L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).	
3.2		Poll interrupts.	
4.1		Set PC .H to Address.H	
4.2		Store as OpCode.	
+X.1			

SBC (F1) Cycles: 5		Size: 2
Direct Indirect Indexed (d),y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (F2) Cycles: 5		Size: 2
Direct Indirect (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer: DB	Store as Operand.
6.1		If D flag == 0: $Result = ui16(A.L) - ui16(Operand) - (1 - C).$ $V\text{ flag} = ((ui16(A.L) \wedge ui16(Operand)) \& (ui16(A.L) \wedge ui8(Result)) \& \$80) \neq 0.$ $C\text{ flag} = Result \leq \$FF.$ $Result = ui8(Result).$ $Z\text{ flag} = Result == \$00.$ $N\text{ flag} = (Result \& \$80) \neq 0.$ If D flag == 1: $Lo = int16(A.L \& 0x0F) - int16(Operand \& 0x0F) - (1 - C);$ if $(Lo < 0)$ $Lo -= 6.$ $BorrowToHi = Lo < 0 ? 1 : 0.$ $HiSum = int16(A.L \gg 4) - int16(Operand \gg 4) - BorrowToHi.$ $V\text{ flag} = (((A.L \gg 4) \wedge (Operand \gg 4)) \& ((A.L \gg 4) \wedge HiSum) \& \$08) \neq 0.$ if $(HiSum < 0)$ $HiSum -= 6.$ $C\text{ flag} = HiSum \geq 0.$ $Result = ui8(((HiSum \& 0x0F) << 4) (Lo \& \$0F)).$ $Z\text{ flag} = Result == \$00.$ $N\text{ flag} = (Result \& \$80) \neq 0.$
6.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (F3) Cycles: 7		Size: 2
Stack Relative Indirect Indexed		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC . Set Address to Pointer + (S .L \$0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank = DB + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: Result = ui16(A .L) - ui16(Operand) - (1 - C). V flag = ((ui16(A .L) ^ ui16(Operand)) & (ui16(A .L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A .L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A .L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A .L >> 4) ^ (Operand >> 4)) & ((A .L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8((((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

PEA (F4)	Cycles: 5	Size: 3
Absolute (Push)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Operand.H.
3.1		Inc. PC .
3.2	Write to ((S .L \$0100)+0)	Push Operand.H onto stack.
4.1		
4.2	Write to ((S .L \$0100)-1)	Push Operand.L onto stack.
5.1		Dec. S .L by 2 and set S .H to 1.
5.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (F5)	Cycles: 4	Size: 2
Direct, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
5.2		Read PC:PB Store as OpCode.
+X.1		

INC (F6)	Cycles: 6	Size: 2
Direct, X (Read/Modify/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand.
2.1		Inc. PC .
2.2		
3.1		If D.L is 0, Address.L = Operand + X.L + D.L , Address.H = D.H , otherwise Address = Operand + X + D . If D.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Perform Operand.L += 1, set N based off (Operand.L & \$80), set Z based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (F7)	Cycles: 6	Size: 2
Direct Indirect Indexed Long [d],y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. PC .
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + D .
3.2	Read Address	Store as Pointer.
4.1		
4.2	If D.L is not 0, read (Address + 1), otherwise read (Address & \$FF00) ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If D.L is not 0, read (Address + 2), otherwise read (Address & \$FF00) ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8((((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
7.2	Read PC:PB	Store as OpCode.
+X.1		

SED (F8)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Discard.
2.1		Set the D flag to 1.
2.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (F9) Cycles: 4		Size: 3
Absolute, Y (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC. Perform Orig = Pointer. Tmp = ui32(Pointer + Y). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

PLX (FA)	Cycles: 4	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		
2.1		
2.2		
3.1		Inc. S.L by 1 and set S.H to 1.
3.2		Store as Operand.L.
4.1	Read u8(S.L +0) \$0100	Set X.L to Operand.L, set N based off (X.L & \$80), and set Z based off X.L .
4.2	Read PC:PB	Store as OpCode.
+X.1		

XCE (FB)	Cycles: 2	Size: 1
Implied		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1	Read PC:PB	Inc. PC .
1.2		Discard.
2.1		Exchanges the Carry and Emulation flags. If Emulation mode is entered, M and X flags are set to 1, S.H is set to 1, and X.H and Y.H are cleared to 0.
2.2		Store as OpCode.
+X.1		

JSR (FC)	Cycles: 8	Size: 3
Absolute Indexed Indirect (a,x) (Jump)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Write to u8(S .L+0) \$0100	Push PC .H onto stack.
3.1		
3.2	Write to u8(S .L-1) \$0100	Push PC .L onto stack.
4.1		Dec. S .L by 2 and set S .H to 1.
4.2	Read PC:PB	Store as Pointer.H.
5.1		Perform Pointer += X .
		Bank = PB .
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Address.L.
7.1		
7.2	Read Pointer + 1:Bank	Store as Address.H.
8.1		Set PC to Address.
8.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (FD) Cycles: 4		Size: 3
Absolute, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform Orig = Pointer. Tmp = ui32(Pointer + X). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = DB + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		If D flag == 0: Result = ui16(A.L) - ui16(Operand) - (1 - C). V flag = ((ui16(A.L) ^ ui16(Operand)) & (ui16(A.L) ^ ui8(Result)) & \$80) != 0. C flag = Result <= \$FF. Result = ui8(Result). Z flag = Result == \$00. N flag = (Result & \$80) != 0. If D flag == 1: Lo = int16(A.L & 0x0F) - int16(Operand & 0x0F) - (1 - C); if (Lo < 0) Lo -= 6. BorrowToHi = Lo < 0 ? 1 : 0. HiSum = int16(A.L >> 4) - int16(Operand >> 4) - BorrowToHi. V flag = (((A.L >> 4) ^ (Operand >> 4)) & ((A.L >> 4) ^ HiSum) & \$08) != 0. if (HiSum < 0) HiSum -= 6. C flag = HiSum >= 0. Result = ui8(((HiSum & 0x0F) << 4) (Lo & \$0F)). Z flag = Result == \$00. N flag = (Result & \$80) != 0.
5.2	Read PC:PB	Store as OpCode.
+X.1		

INC (FE)	Cycles: 7	Size: 3
Absolute, X (Read/Modiy/Write)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \mathbf{X})$. Pointer = $\text{ui16}(\text{Tmp})$. Bank = $\mathbf{DB} + (\text{Tmp} \gg 16)$.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Perform $\text{Operand.L} += 1$, set N based off ($\text{Operand.L} \& \$80$), set Z based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read PC:PB	Store as OpCode.
+X.1		

SBC (FF)	Cycles: 5	Size: 4
Absolute Long, X (Read)		
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. PC .
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. PC .
2.2	Read PC:PB	Store as Pointer.H.
3.1		Inc. PC .
3.2	Read PC:PB	Stores as Bank.
4.1		Inc. PC . Perform $\text{Tmp} = \text{ui32}(\text{Pointer} + \text{X})$. $\text{Pointer} = \text{ui16}(\text{Tmp})$. $\text{Bank} += (\text{Tmp} \gg 16)$.
4.2	Read Pointer:Bank	Store as Operand.
5.1		If D flag == 0: $\text{Result} = \text{ui16}(\text{A.L}) - \text{ui16}(\text{Operand}) - (1 - \text{C})$. V flag = $((\text{ui16}(\text{A.L}) \wedge \text{ui16}(\text{Operand})) \& (\text{ui16}(\text{A.L}) \wedge \text{ui8}(\text{Result}) \& \$80) \neq 0)$. C flag = $\text{Result} \leq \$\text{FF}$. $\text{Result} = \text{ui8}(\text{Result})$. Z flag = $\text{Result} == \$00$. N flag = $(\text{Result} \& \$80) \neq 0$. If D flag == 1: $\text{Lo} = \text{int16}(\text{A.L} \& 0x0F) - \text{int16}(\text{Operand} \& 0x0F) - (1 - \text{C})$; if $(\text{Lo} < 0)$ $\text{Lo} -= 6$. $\text{BorrowToHi} = \text{Lo} < 0 ? 1 : 0$. $\text{HiSum} = \text{int16}(\text{A.L} \gg 4) - \text{int16}(\text{Operand} \gg 4) - \text{BorrowToHi}$. V flag = $((\text{A.L} \gg 4) \wedge (\text{Operand} \gg 4)) \& ((\text{A.L} \gg 4) \wedge \text{HiSum}) \& \$08 \neq 0$. if $(\text{HiSum} < 0)$ $\text{HiSum} -= 6$. C flag = $\text{HiSum} \geq 0$. $\text{Result} = \text{ui8}(((\text{HiSum} \& 0x0F) \ll 4) \mid (\text{Lo} \& \$0F))$. Z flag = $\text{Result} == \$00$. N flag = $(\text{Result} \& \$80) \neq 0$.
5.2	Read PC:PB	Store as OpCode.
+X.1		